

小结

C++为解决某些特殊问题设置了一系列特殊的处理机制。

有的程序需要精确控制内存分配过程，它们可以通过在类的内部或在全局作用域中自定义 `operator new` 和 `operator delete` 来实现这一目的。如果应用程序为这两个操作定义了自己的版本，则 `new` 和 `delete` 表达式将优先使用应用程序定义的版本。

有的程序需要在运行时直接获取对象的动态类型，运行时类型识别 (RTTI) 为这种程序提供了语言级别的支持。RTTI 只对定义了虚函数的类有效；对没有定义虚函数的类，虽然也可以得到其类型信息，但只是静态类型。

当我们定义指向类成员的指针时，在指针类型中包含了该指针所指成员所属类的类型信息。成员指针可以绑定到该类当中任意一个具有指定类型的成员上。当我们解引用成员指针时，必须提供获取成员所需的对象。

C++定义了另外几种聚集类型：

- 嵌套类，定义在其他类的作用域中，嵌套类通常作为外层类的实现类。
- `union`，是一种特殊的类，它可以定义几个数据成员但是在任意时刻只有一个成员有值，`union` 通常嵌套在其他类的内部。
- 局部类，定义在函数的内部，局部类的所有成员都必须定义在类内，局部类不能含有静态数据成员。

C++支持几种固有的不可移植的特性，其中位域和 `volatile` 使得程序更容易访问硬件；链接指示使得程序更容易访问用其他语言编写的代码。

术语表

匿名 union (anonymous union) 未命名的 `union`，不能用于定义对象。匿名 `union` 的成员也是外层作用域的成员。匿名 `union` 不能包含成员函数，也不能包含私有成员或受保护的成员。

位域 (bit-field) 特殊的类成员，该成员含有一个整型值以指定为其分配的二进制位数。如果可能的话，在类中连续定义的位域将被压缩在一个普通的整数值当中。

判别式 (discriminant) 是一种使用一个对象判断 `union` 的当前值类型的编程技术。

dynamic_cast 是一个运算符，执行从基类向派生类的带检查的强制类型转换。当基类中至少含有一个虚函数时，该运算符负责检查指针或引用所绑定的对象的动态类型。如果对象类型与目标类型（或其派生类）一致，则类型转换完成。否则，指针

转换将返回一个值为 0 的指针；引用转换将抛出一个异常。

枚举类型 (enumeration) 将一组整型常量命名后聚合在一起形成的类型。

枚举成员 (enumerator) 是枚举类型的成员。枚举成员是常量，可以用在任何需要整型常量的地方。

free 是定义在 `cstdlib` 中的低层函数，负责释放内存。`free` 只能释放由 `malloc` 分配的内存。

链接指示 (linkage directive) 支持 C++ 程序调用其他语言编写的函数的一种机制。所有编译器都应支持调用 C++ 和 C 函数，至于是否支持其他语言则由编译器决定。

局部类 (local class) 定义在函数中的类。局部类只有在其外层函数内可见。局部类

的所有成员都必须定义在类的内部。局部类不能含有静态成员。局部类成员不能访问外层函数的非静态变量，只能访问类型名字、静态变量或枚举成员。

malloc 是定义在 `cstdlib` 中的低层函数，负责分配内存。**malloc** 分配的内存必须由 **free** 释放。

mem_fn 是一个标准库类模板，根据指向成员函数的指针生成一个可调用对象。

嵌套类 (nested class) 定义在其他类内部的类，嵌套类定义在它的外层作用域中：在外层类的作用域中嵌套类的名字必须唯一，在外层类之外可以被重用。在外层类之外访问嵌套类需要用作用域运算符指明嵌套类所属的范围。

嵌套类型 (nested type) “嵌套类”的同义词。

不可移植 (nonportable) 固有的与机器有关的特性，当程序转移到其他机器或编译器上时需要修改代码。

operator delete 是一个标准库函数，用于释放由 **operator new** 分配的未指明类型的、未构造的内存空间。相应的，**operator delete[]** 释放由 **operator new[]** 为数组分配的内存。

operator new 是一个标准库函数，用于分配一个给定大小的、未指明类型的、未构造的内存空间。标准库函数 **operator new[]** 为数组分配原始内存。与 **allocator** 类相比，这两个标准库函数提供的内存分配机制更低级。现代的 C++ 程序应该使用 **allocator** 而不是这两个函数。

定位 new 表达式 (placement new expression) 是 **new** 的一种特殊形式，在给定的内存中构造对象。它不分配内存，而是根据实参指定在哪儿构造对象。它是对 **allocator** 类的 **construct** 成员的行为的一种低级模拟。

成员指针 (pointer to member) 其中既包含类类型，也包含指针所指的成员类型。

成员指针的定义必须同时指定类的名字以及指针所指的成员类型：

```
T C::*pmem = &C::member;
```

该语句将 **pmem** 定义为一个指针，它可以指向类 **C** 的成员，并且该成员的类型是 **T**，然后初始化 **pmem** 令其指向类 **C** 的名为 **member** 的成员。要使用该指针，我们必须提供 **C** 的一个对象或指针：

```
classobj.*pmem;
```

```
classptr->*pmem;
```

从 **classptr** 所指的对象 **classobj** 中获取 **member**。

运行时类型识别 (run-time type identification) 是 C++ 的一种特性，允许在运行时获取指针或引用的动态类型。**RTTI** 运算符包括 **typeid** 和 **dynamic_cast**，为含有虚函数的类的指针或引用提供动态类型。当作用于其他类型时，返回的结果是指针或引用的静态类型。

限定作用域的枚举类型 (scoped enumeration) 是一种新的枚举类型，它的枚举成员不能被外层作用域直接访问。

typeid 运算符 (typeid operator) 是一个一元运算符，返回标准库类型 **type_info** 的引用，表示给定表达式的类型。当表达式是某个含有虚函数的类型的对象时，返回表达式的动态类型；此类表达式在运行时求值。如果表达式的类型是指针、引用或其他未定义虚函数的类型，则返回指针、引用或对象的静态类型；此类表达式不会被求值。

type_info **typeid** 运算符返回的标准库类型。**type_info** 的细节因机器而异，但是必须提供一组操作，其中名为 **name** 的函数负责返回一个表示类型名字的字符串。**type_info** 对象不能被拷贝、移动或赋值。

联合 (union) 是一种和类有些相似的类型。可以包含多个数据成员，但是同一时刻只能有一个成员有值。联合可以有包括

构造函数和析构函数在内的成员函数。联合不能被用作基类。在 C++11 新标准中，联合可以含有类类型的成员，前提是这些类自定义了拷贝控制成员。对于这样的联合来说，如果它们没有定义自己的拷贝控制成员，则编译器将为它们生成删除的版本。

不限定作用域的枚举类型（unscoped enumeration） 该枚举类型的枚举成员在枚举类型的外层作用域中可以访问。

volatile 是一种类型限定符，告诉编译器变量可能在程序的直接控制之外发生改变。它起到一种标示的作用，令编译器不对代码进行优化操作。

附录 A

标准库

内容

A.1 标准库名字和头文件	766
A.2 算法概览	770
A.3 随机数	781

本附录介绍了标准库中算法和随机数部分的一些额外细节。这里还提供了一个我们使用过的所有标准库名字的列表，列表中给出了每个名字所在的头文件。

在第 10 章中我们使用过一些较常用的算法，并且描述了算法之下的架构。在本附录中，我们将列出所有标准库算法，按它们执行的操作的种类来组织。

在 17.4 节（第 660 页）中我们描述了随机数库的架构，并使用了几个分布类型。库中定义了若干随机数引擎和 20 种不同的分布。在本附录中，我们将列出所有引擎和分布类型。